

CS4M: Event-Arrival Online Provenance Intrusion Detection with Compact Semantic State

Anonymous Submission

Abstract

Provenance-based intrusion detection systems (PIDSs) are effective because system provenance exposes causal relationships among processes, files, and network connections. Yet many systems that process audit logs online still make detection decisions over a derived object: a completed graph, a time window, a segmented task graph, or a periodically reconstructed provenance structure. This creates a gap between online log collection and online alert semantics. For deployment, defenders often need a stricter property: when a provenance event arrives, the detector should score that event immediately, emit evidence if it is suspicious, and only then update its state.

We present CS4M, a lightweight semantic-state detector for event-arrival online provenance intrusion detection. CS4M replaces full graph materialization with a compact causal semantic-state table. For each arriving event, CS4M reads pre-update source and destination states, selects an E4 dual semantic target case, applies the corresponding low-rank prediction head, converts the residual into a calibrated alert score, emits event-level and node-level evidence, and updates memory only after scoring. This score-before-update order prevents an event from explaining itself through the state it just created, while online finite state memory bounds active state storage.

Our current evidence lock covers Linux, FreeBSD, Android, and Windows provenance settings. It records event/node alert quality, online memory and throughput sources, and semantic-audit risks without using test labels for runtime decisions. Across the locked artifacts, CS4M produces event-level alerts and node evidence with primary online RSS in the tens to low hundreds of MB and stream-processing rates of thousands of events per second. Several numbers remain deliberately marked for final reconciliation, including ClearScope E5 top- k under a 51-GT canonical-aware node mapping and some runtime-source normalization. These markers are kept visible to avoid turning post-hoc audits or user-provided table locks into unsupported manuscript claims.

1 Introduction

Whole-system provenance has become a central abstraction for intrusion detection and investigation. By recording how processes interact with files, sockets, and other system objects, provenance logs let defenders reconstruct long-range attack behavior that is invisible to short local event windows. This is why systems such as HOLMES, UNICORN, KAIROS, FLASH, and ORTHRUS have made substantial progress on APT detection, triage, graph representation learning, and attribution quality [2, 1, 4, 6, 7].

However, the operational question addressed in this paper is narrower and more stringent than provenance investigation: *when can an alert be raised?* Many PIDS pipelines are online in the sense that they ingest or process logs continuously, but their detection semantics are still tied to a derived object. They may require graph construction, window completion, task-guided segmentation, repeated-edge pruning, graph embedding, or periodic inference before the detector can decide what to report. These are valid design choices for investigation and attribution, but they create what we call *pseudo-real-time* alerting: the log stream is online, while the alerting unit is not the event that just arrived.

CS4M targets a stricter deployment model. The detector receives provenance events in stream order and must decide on the current event before seeing future events or using labels. It cannot wait for a future graph window to become complete, and it cannot let the current event update memory before being scored. The central claim of CS4M is therefore not that graphs are unhelpful, nor that attribution systems are unnecessary. The claim is that immediate alerting needs a different computational object: a compact online semantic state rather than a materialized provenance graph.

CS4M implements this model with score-before-update residual semantics. Each process, file, or network node is represented by runtime-visible semantic tokens and a learned embedding. During online inference, CS4M keeps a bounded state table for active nodes. For an arriving event, it reads the pre-update source and destination states, chooses one of two E4 target cases, scores the residual with the corresponding target-case-specific low-rank head, emits event and node evidence, and only then writes the event into memory. The normal stateful case predicts event semantics from existing causal context; the both-cold case falls back to an action-aware semantic target when neither endpoint has useful state. Online finite state memory (OFSM) bounds active state storage by separating logical nodes from physical state slots.

This framing also changes how the paper evaluates evidence. Ground truth is used only after streaming inference is complete. Runtime thresholds and suppression decisions must come from validation data or runtime-visible fields, not test labels, attack windows, or test rankings. Our PaperSpine evidence lock records this boundary explicitly. Current locked results cover THEIA E3, CADETS E3, ClearScope E3/E5, THEIA E5, and OPTC 051, with per-dataset event and node alert metrics, online memory/throughput provenance, and semantic audit notes. Where a result is not yet safe to write as a final claim, the manuscript marks it as such rather than hiding the uncertainty.

Contributions. This paper makes four contributions:

- We define *event-arrival online PIDS*, a stricter alerting

model in which the current provenance event is the inference unit and must be scored before it updates detector state.

- We design CS4M, a compact semantic-state detector that replaces graph/window inference with score-before-update residual prediction over runtime-visible process, file, and network semantics.
- We introduce the E4 dual semantic scoring path: target-case-specific low-rank heads for stateful event semantics and both-cold action semantics, combined with OFSM for bounded active state storage.
- We build an evidence-grounded evaluation plan across Linux, FreeBSD, Android, and Windows datasets, including explicit markers for unresolved ClearScope E5 51-GT top- k reconciliation and runtime-source normalization.

2 Background and Motivation

2.1 Provenance Graphs Are Useful but Not Always the Right Alert Unit

A provenance graph connects system entities and activities through observed causal events. This abstraction is powerful for investigation: once a graph is available, analysts can follow information flow, reconstruct attack paths, and attribute suspicious activity to earlier causes. Graph-centered PIDSs exploit this structure in different ways. HOLMES correlates suspicious information flows into higher-level attack graphs [2]. UNICORN studies runtime provenance detection under graph growth [1]. KAIROS and FLASH use whole-system provenance and graph representation learning for practical intrusion detection and investigation [4, 6]. ORTHRUS focuses on the quality of attribution, a problem that remains important after alerts are generated [7].

These systems address real problems, but they optimize a different alerting unit than CS4M. A graph, segment, or window must first be accumulated or constructed. The detector may then score nodes, edges, subgraphs, or reconstructed attack structures. That path is appropriate when the objective is rich investigation context. It is less appropriate when the objective is to decide on every incoming event under endpoint deployment budgets.

2.2 From Online Processing to Online Alert Semantics

We distinguish online processing from online alert semantics. A system can consume logs continuously while still delaying alert decisions until a graph window, segment, or periodic batch is ready. CS4M instead requires the current event to be the decision point. The model can maintain state from the past, but it cannot use future events, future graph completeness, test labels, or post-hoc rankings to decide whether the current event alerts.

This distinction is the paper’s main motivation. The goal is not to discard provenance graphs. Graph reconstruction remains useful after an alert, and attribution-oriented systems

remain complementary. The goal is to make the first alert deployable: fast, memory-bounded, label-free at runtime, and accompanied by evidence that an analyst can inspect immediately.

3 Problem Statement and Online Detection Model

Modern provenance-based intrusion detection systems (PIDSs) use system audit logs to recover causal relationships among processes, files, and network connections. This line of work is effective for post-hoc investigation and APT reconstruction because provenance graphs preserve long-range dependencies that short local event windows often miss [1, 2, 4]. The deployment problem targeted by CS4M is narrower: online alerting over a real-time provenance stream. In this setting, the detector must decide whether the current event is suspicious when the event arrives, under endpoint resource budgets.

3.1 Pseudo-Real-Time PIDS

We use *pseudo-real-time* to describe systems whose log collection or periodic processing is online, but whose alert semantics are tied to a derived object rather than the arriving event. A provenance graph is not a primitive observation; it is constructed from accumulated logs. Before graph-based inference can run, a system may need a graph window, task segment, pruning pass, edge featurization stage, embedding update, or periodic inference round. These steps are often reasonable for investigation and attribution, but they make the alert depend on the completion of a graph/window/segment rather than on the arrival of the current event.

This definition is intentionally precise. We do not claim that graph-first PIDSs cannot ingest logs online, or that graph reconstruction is unnecessary. We claim that the *alerting unit* is different. If a system must first materialize a windowed graph or run periodic graph inference, then the suspicious event is not itself the inference unit. In contrast, CS4M targets event-arrival online detection: e_t must be scored using only information available before or at time t , and the alert decision for e_t must be emitted before future events are observed.

3.2 Online Intrusion Detection Model

We assume a host emits a time-ordered stream of provenance events:

$$\mathcal{E} = \{e_1, e_2, \dots, e_T\}. \quad (1)$$

Each event is

$$e_t = (s_t, d_t, a_t, \tau_t, c_t^s, c_t^d), \quad (2)$$

where s_t is the source node, d_t is the destination node, a_t is the action type, τ_t is the timestamp, and c_t^s, c_t^d are endpoint types. The detector receives e_t once, in stream order, and must produce an event-level decision:

$$\text{alert}_t \in \{0, 1\}, \quad (3)$$

plus node-level evidence for analyst triage. It cannot wait for a future graph window to become complete.

The online decision must satisfy two causality constraints. First, the detector may use only current and past runtime-visible fields, such as node identifiers, types, action labels, timestamps, paths, process names, commands, and network attributes. It may not use future events, complete-test statistics, full-test rankings, attack windows, or ground-truth labels. Second, the detector must score before update: the state used to explain e_t must be the state that existed before e_t was written.

The defender controls the logging mechanism, training data collection, and detector deployment. Training data is assumed benign or cleaned according to the evaluation protocol. The attacker may execute processes, access files, and communicate over the network, thereby producing provenance events, but cannot tamper with the audit stream or detector state. Attacks not captured by the logging substrate, such as hardware side channels or log-suppression attacks against the trusted collector, are outside the scope.

3.3 Evaluation Boundary

Ground truth is not part of the runtime detector. CS4M may use training data, validation data, and runtime-visible event fields to learn embeddings, prediction heads, thresholds, and suppression caches. Ground-truth malicious nodes, attack windows, and labels are attached only after streaming inference has emitted alerts. This separation is important because a detector can appear online while still leaking future or label-derived information into thresholding or alert selection. CS4M’s paper tables therefore distinguish runtime outputs from post-stream evaluation artifacts.

3.4 Design Objective

Our objective is to build a detector that satisfies four requirements simultaneously:

- **Real-time alerting:** process each event when it arrives, without requiring pre-built provenance graphs or fixed detection windows.
- **Lightweight deployment:** run on resource-limited hosts without GPU inference, large resident graphs, or heavy-weight GNN models.
- **High-accuracy semantic generalization:** detect deviations from normal causal semantics without relying on brittle hand-written rules.
- **Explainable evidence:** expose the event target, residual score, calibration group, and node evidence that caused an alert.

CS4M addresses this objective by replacing graph materialization with a compact causal semantic-state table. Rather than asking whether a subgraph looks abnormal after a window is constructed, CS4M asks whether the current event is semantically predictable from the state that existed before the event.

4 CS4M Design

CS4M consists of three modules, shown conceptually in Fig. 2: (I) runtime-visible semantic modeling, (II) causal semantic-state memory, and (III) E4 dual residual scoring. The modules operate in a single pass over the event stream. The design is organized around five deployment constraints:

- **No completed graph.** CS4M keeps compact node state rather than a materialized provenance graph.
- **No future context.** The current event is scored from pre-update source and destination states.
- **No label-derived runtime policy.** Thresholds and suppression use validation data and runtime-visible fields; labels are attached only after the stream.
- **Low-rank CPU scoring.** E4 dual uses target-case-specific low-rank heads rather than graph neural inference.
- **Bounded active memory.** OFSM stores active physical state slots instead of all historical node states.

4.1 Data-Flow Perspective Log Semantic Modeling

CS4M first abstracts raw logs into provenance events and semantic tokens. For a node v , let

$$\mathcal{T}(v) = \{w_1, w_2, \dots, w_m\} \quad (4)$$

be the token set extracted from process, file, or network attributes. Process tokens can include executable names and command tags; file tokens can include path components and extensions; network-flow tokens can include address scope, port bucket, and protocol. Action types are also tokenized.

Let $\phi(w) \in \mathbb{R}^d$ be a Word2Vec embedding. CS4M computes node embeddings by mean pooling:

$$\mathbf{e}_v = \frac{1}{|\mathcal{T}(v)|} \sum_{w \in \mathcal{T}(v)} \phi(w), \quad (5)$$

and action embeddings as

$$\mathbf{e}_a = \phi(a). \quad (6)$$

For event e_t , the semantic inputs are

$$\mathbf{e}_{s_t}, \mathbf{e}_{d_t}, \mathbf{e}_{a_t} \in \mathbb{R}^{64}. \quad (7)$$

This representation lets CS4M exploit semantic regularities in logs without materializing a graph neighborhood.

4.2 Causal Semantic-Structural State-Space Model

CS4M maintains a state for nodes that are currently relevant to online scoring. The state update is semantic because the written message is derived from the source, action, and destination embeddings, and structural because the state is indexed by

provenance nodes and read before each event is scored. For event e_t , the write message is

$$\mathbf{z}_t = \frac{\mathbf{e}_{s_t} + \mathbf{e}_{a_t} + \mathbf{e}_{d_t}}{3}. \quad (8)$$

Action semantics are retained in the state update because the action describes how the event changes the system state. In E4 dual, action semantics are also used as a prediction target for the both-cold case, where neither endpoint has a useful pre-update state.

The general continuous-time memory model is

$$\frac{d\mathbf{h}(t)}{dt} = \mathbf{A}\mathbf{h}(t) + \mathbf{B}\mathbf{u}(t), \quad (9)$$

$$\mathbf{y}(t) = \mathbf{C}\mathbf{h}(t) + \mathbf{D}\mathbf{u}(t). \quad (10)$$

Under zero-order hold on $[t_k, t_{k+1})$, with $\Delta_k = t_{k+1} - t_k$, the discrete update is

$$\mathbf{h}_{k+1} = \bar{\mathbf{A}}_k \mathbf{h}_k + \bar{\mathbf{B}}_k \mathbf{u}_k, \quad (11)$$

where

$$\bar{\mathbf{A}}_k = e^{\Delta_k \mathbf{A}}, \quad (12)$$

$$\bar{\mathbf{B}}_k = \int_0^{\Delta_k} e^{(\Delta_k - \tau)\mathbf{A}} \mathbf{B} d\tau. \quad (13)$$

CS4M uses efficient diagonal or near-diagonal state dynamics. A fixed EMA instance is

$$\mathbf{h}_v^{t+1} = (1 - \lambda)\mathbf{h}_v^t + \lambda\mathbf{z}_t, \quad 0 < \lambda < 1. \quad (14)$$

An S4D instance uses complex eigenvalues $\lambda_i = -\alpha_i + j\omega_i$:

$$\bar{A}_i = e^{-\Delta\alpha_i} (\cos(\Delta\omega_i) + j \sin(\Delta\omega_i)). \quad (15)$$

The real state is obtained by concatenating real and imaginary parts.

4.3 OFSM: Online Finite State Memory

In a long-running log stream, the set of historical nodes grows continuously:

$$\mathcal{V}_{1:t} = \{v \mid v \text{ appears in an event before or at } t\}. \quad (16)$$

If CS4M permanently stored a dense state vector $\mathbf{h}_v^t \in \mathbb{R}^d$ for every historical node, the state memory would be

$$\mathcal{M}_{\text{dense}}(t) = |\mathcal{V}_{1:t}| \cdot d \cdot b, \quad (17)$$

where b is the number of bytes per floating-point value. Since $|\mathcal{V}_{1:t}|$ can grow for as long as the host runs, dense historical state is not a deployable online design.

OFSM bounds this memory by separating *logical nodes* from *physical state slots*. Let $\mathcal{A}_t \subseteq \mathcal{V}_{1:t}$ be the active node set. CS4M maintains only recently active nodes, with

$$|\mathcal{A}_t| \leq K_s, \quad (18)$$

where K_s is the active-state capacity. A node evicted from $\mathcal{A}_t$ is treated as cold when it appears again; its context falls back to the static embedding $\mathbf{e}_v$ rather than restoring an old state. The active memory is therefore bounded by

$$\mathcal{M}_{\text{active}} \leq K_s \cdot d \cdot b. \quad (19)$$

OFSM further compresses active states through a mapping

$$\pi_t(v) \in \{1, \dots, P_t\}, \quad v \in \mathcal{A}_t, \quad (20)$$

where P_t is the number of physical state slots. The physical state table is

$$\mathbf{S}_t = [\mathbf{s}_1^t, \mathbf{s}_2^t, \dots, \mathbf{s}_{P_t}^t], \quad \mathbf{s}_p^t \in \mathbb{R}^d. \quad (21)$$

The readable state of node v is

$$\mathbf{h}_v^t = \mathbf{s}_{\pi_t(v)}^t. \quad (22)$$

If several nodes map to the same slot p , they form an online state cluster:

$$\mathcal{C}_p^t = \{v \in \mathcal{A}_t \mid \pi_t(v) = p\}. \quad (23)$$

Since $P_t \leq |\mathcal{A}_t| \leq K_s$, OFSM memory is

$$\mathcal{M}_{\text{OFSM}}(t) = P_t \cdot d \cdot b + \mathcal{M}_{\text{index}} = O(P_t d) \leq O(K_s d), \quad (24)$$

where $\mathcal{M}_{\text{index}}$ is the node-to-slot mapping overhead. The compression ratio is

$$\rho_t = \frac{|\mathcal{A}_t|}{P_t}, \quad (25)$$

and the relative reduction in state-vector memory is

$$\Delta_{\text{mem}}(t) = 1 - \frac{P_t}{|\mathcal{A}_t|}. \quad (26)$$

State signatures and merging. OFSM does not compare all raw states directly. For each physical slot p , it computes a signature

$$\phi_p^t = \Phi(\mathbf{s}_p^t). \quad (27)$$

The time-domain signature is

$$\phi_p^t = \frac{\mathbf{s}_p^t}{\|\mathbf{s}_p^t\|_2 + \epsilon}. \quad (28)$$

A frequency-domain signature can instead use the discrete Fourier transform:

$$\phi_p^t = \frac{|\mathcal{F}(\mathbf{s}_p^t)|}{\|\mathcal{F}(\mathbf{s}_p^t)\|_2 + \epsilon}. \quad (29)$$

The slot similarity is

$$\text{sim}(p, q) = \frac{\langle \phi_p^t, \phi_q^t \rangle}{\|\phi_p^t\|_2 \|\phi_q^t\|_2 + \epsilon}. \quad (30)$$

OFSM may merge two slots when

$$\text{type}(p) = \text{type}(q), \quad \text{sim}(p, q) \geq \tau_m, \quad n_p \geq n_{\min}, \quad n_q \geq n_{\min}, \quad (31)$$

where n_p, n_q are access counts and τ_m is the merge threshold.

When slots p and q are merged, OFSM replaces slot p by the count-weighted average

$$\mathbf{s}_{p \leftarrow q}^t = \frac{n_p \mathbf{s}_p^t + n_q \mathbf{s}_q^t}{n_p + n_q}, \quad (32)$$

updates $n_{p \leftarrow q} = n_p + n_q$, remaps all nodes in $\mathcal{C}_p^t \cup \mathcal{C}_q^t$ to slot p , and releases slot q so that $P_t \leftarrow P_t - 1$.

Copy-on-write isolation. State sharing must not let one node-specific update pollute every node in a cluster. If node v maps to a shared slot p with $|\mathcal{C}_p^t| > 1$, OFSM materializes a private copy before updating:

$$\mathbf{s}_{p'}^t = \mathbf{s}_p^t, \quad \pi_t(v) = p', \quad \mathcal{C}_p^t \leftarrow \mathcal{C}_p^t \setminus \{v\}. \quad (33)$$

Only the new slot is updated:

$$\mathbf{s}_{p'}^{t+1} = \text{Update}(\mathbf{s}_{p'}^t, \mathbf{z}_t). \quad (34)$$

Thus OFSM changes how states are stored and shared, but not the event message in Eq. 8 or the score-before-update order.

4.4 E4 Dual State-Aware Semantic Prediction

When event e_t arrives, CS4M reads the pre-update states $\mathbf{h}_{s_t}^{t-}$ and $\mathbf{h}_{d_t}^{t-}$. Let

$$I_s = \text{has_state}(s_t, t), \quad I_d = \text{has_state}(d_t, t). \quad (35)$$

The endpoint representations are

$$\mathbf{r}_{s_t} = \begin{cases} (\mathbf{h}_{s_t}^{t-} + \mathbf{e}_{s_t})/2, & I_s = 1, \\ \mathbf{e}_{s_t}, & I_s = 0, \end{cases} \quad (36)$$

and

$$\mathbf{r}_{d_t} = \begin{cases} (\mathbf{h}_{d_t}^{t-} + \mathbf{e}_{d_t})/2, & I_d = 1, \\ \mathbf{e}_{d_t}, & I_d = 0. \end{cases} \quad (37)$$

With endpoint-type one-hot vectors $\mathbf{o}_{c_t^s}, \mathbf{o}_{c_t^d} \in \mathbb{R}^4$, the context is

$$\mathbf{x}_t = [\mathbf{r}_{s_t}; \mathbf{r}_{d_t}; \mathbf{o}_{c_t^s}; \mathbf{o}_{c_t^d}] \in \mathbb{R}^{136}. \quad (38)$$

E4 dual separates scoring into two target cases:

$$\kappa_t = \begin{cases} \text{event_semantic_target}, & I_s = 1 \text{ or } I_d = 1, \\ \text{both_cold_action_target}, & I_s = 0 \text{ and } I_d = 0. \end{cases} \quad (39)$$

The first case covers ordinary stateful inference: at least one endpoint has pre-update causal memory, so the detector asks whether the event semantics are consistent with that memory. The second case covers cold-start events: neither endpoint has useful memory, so the detector uses an action-aware target rather than pretending that absent state can explain the event.

For the stateful case, the semantic target is the event embedding:

$$\mathbf{y}_t^{\text{event}} = \frac{\mathbf{e}_{s_t} + \mathbf{e}_{a_t} + \mathbf{e}_{d_t}}{3}. \quad (40)$$

For the both-cold case, CS4M uses an action-aware semantic target with the same embedding dimension:

$$\mathbf{y}_t^{\text{cold}} = \mathbf{e}_{a_t}. \quad (41)$$

This distinction matters because a residual that is normal for a well-known process-file interaction may be suspicious when both endpoints are unseen. The both-cold branch asks a narrower question: whether the action itself is predictable from the event’s typed semantic context.

The selected target is therefore

$$\mathbf{y}_t = \begin{cases} \mathbf{y}_t^{\text{event}}, & \kappa_t = \text{event_semantic_target}, \\ \mathbf{y}_t^{\text{cold}}, & \kappa_t = \text{both_cold_action_target}. \end{cases} \quad (42)$$

E4 dual uses separate low-rank parameters for the two target cases:

$$\hat{\mathbf{y}}_t = \begin{cases} \mathbf{x}_t \mathbf{W}_1^e \mathbf{W}_2^e + \mathbf{b}^e, & \kappa_t = \text{event_semantic_target}, \\ \mathbf{x}_t \mathbf{W}_1^c \mathbf{W}_2^c + \mathbf{b}^c, & \kappa_t = \text{both_cold_action_target}, \end{cases} \quad (43)$$

where each $\mathbf{W}_1 \in \mathbb{R}^{136 \times r}$, each $\mathbf{W}_2 \in \mathbb{R}^{r \times 64}$, and each bias is in $\mathbb{R}^{64}$. The two heads share the same context construction and state-update order, but do not share the final low-rank prediction parameters. This is the core E4 dual method used in the paper: it handles normal stateful events and both-cold events as different semantic prediction problems without requiring graph construction.

4.5 Residual Scoring and Threshold Calibration

Given the target selected by Eq. 39 and prediction $\hat{\mathbf{y}}_t$, CS4M computes a residual distance. With cosine distance,

$$\ell_t = 1 - \frac{\mathbf{y}_t^\top \hat{\mathbf{y}}_t}{\|\mathbf{y}_t\|_2 \|\hat{\mathbf{y}}_t\|_2 + \epsilon}. \quad (44)$$

Using validation losses, the empirical tail probability is

$$p_t = \frac{1 + |\{i : \ell_i^{\text{val}} \geq \ell_t\}|}{1 + N_{\text{val}}}, \quad (45)$$

and the residual score is

$$\sigma_t = -\log p_t. \quad (46)$$

Scores are calibrated by group:

$$g_t = (\kappa_t, a_t, c_t^s, c_t^d), \quad (47)$$

with fallback levels that remove endpoint types and then action type when validation support is sparse. If group g has at least $N_{\min}$ validation samples,

$$\theta_g = Q_g(\{\sigma_i^{\text{val}} : g_i = g\}). \quad (48)$$

Sparse groups use a conservative fallback:

$$\theta_g = \max(\theta_{\text{parent}}, \theta_{\text{global}}, \max(\sigma_i^{\text{val}} : g_i = g) + m). \quad (49)$$

An event becomes an alert candidate when $\sigma_t \geq \theta_{g_t}$.

Field group	Representative columns in <code>online_event_alerts</code> stream, in order.
Event order	<code>stream_pos</code> , <code>event_index</code> , <code>timestamp_ns</code>
Event identity	<code>src_idx</code> , <code>dst_idx</code> , <code>action</code> , <code>src_type</code> , <code>dst_type</code>
Score evidence	<code>event_score</code> , <code>threshold</code> , <code>residual_score</code> , <code>residual_cos</code>
Calibration	<code>target_case</code> , <code>threshold_level</code> , <code>threshold_group_key</code>
Support policy	<code>validation_group_count</code> , <code>low_support_policy</code>
Suppression	<code>endpoint_pair_key</code> , <code>endpoint_suppression</code>
State evidence	<code>src_state_norm</code> , <code>dst_state_norm</code> , <code>src_cluster_id</code> , <code>dst_cluster_id</code>

Table 1: Alert evidence emitted by the online scoring path. The table lists actual columns observed in current E4 output artifacts. These fields are runtime outputs; labels are attached later for evaluation.

5 Real-Time Alerting and Explainability

CS4M applies pair-only suppression to reduce repeated benign alerts. Let

$$k_t = (s_t, d_t, a_t). \quad (50)$$

If $C(k_t) \geq K$ in train and validation data and

$$\sigma_t < \theta_{g_t} + \delta, \quad (51)$$

then CS4M suppresses the event alert. The final decision is

$$\text{alert}_t = \mathbb{I}(\sigma_t \geq \theta_{g_t})(1 - \text{suppress}_t). \quad (52)$$

Endpoint-only suppression is deliberately excluded because it can suppress true-positive evidence associated with suspicious endpoints.

The key distinction from graph-first PIDSs is that CS4M performs inference over the event stream itself. The detector does not wait for graph completeness, does not require repeated-operation pruning before inference, and does not run a GNN over a materialized graph. Its per-event cost is dominated by low-rank prediction:

$$O(dr), \quad (53)$$

where $d = 64$ is the embedding/state dimension and r is the low-rank dimension. E4 dual uses two such low-rank heads, one for each target case, so the constant factor remains small. OFSM changes the state memory from $O(|V_{1:t}|d)$ to $O(P_t d) + \mathcal{M}_{\text{index}}$ with $P_t \leq K_s$, so the memory footprint is controlled by active physical states rather than historical node count. The deployment memory consists of OFSM state slots, compact prediction heads, calibration tables, suppression cache, and embedding LRU cache. This design supports CPU-only real-time inference on resource-constrained hosts.

6 Implementation

The implementation follows the event-arrival model rather than a batch graph pipeline. Dataset runners materialize or reuse split event indices, load compact node/action embeddings, load the Phase3E state checkpoint and the Phase3G E4 conditional head, and then execute the test

runner for the current THEIA E3 E4 artifact, for example, fixes `RUN_ONLY=E4_NONE`, uses `SSPM_SCORE_HEAD=conditional_action_semantic`, loads an E4 conditional head checkpoint, enables lazy memmap-backed embedding lookup, and keeps endpoint suppression in pair-only mode. These settings are implementation evidence for the paper’s main path.

Streaming ingestion and state access. The scoring loop reads the event index sequentially. For each row, it resolves source, destination, action, and endpoint types, then reads the source and destination states before computing the score. If the fast path batches several rows for vectorized prediction, the per-row context is still constructed from the pre-update state available at that row, and the alert record preserves `stream_pos` and `event_index` for audit. Post-stream label attachment and coverage reports are generated after online outputs have already been written.

Lazy embeddings and bounded memory. The deployment path uses compact node embeddings backed by memmap and an LRU cache, avoiding a requirement to load every historical node embedding into RAM. OFSM limits active state memory through the logical-node to physical-slot mapping described in Sec. 4.3. Runtime memory reports therefore distinguish online scoring memory from post-stream evaluation memory.

Portable configuration. The implementation uses environment variables for machine-specific settings such as result roots, cache roots, Python interpreter, and database connection parameters. The paper treats absolute paths in historical result metadata as artifact provenance, not deployment requirements. This is important for reproducibility on a new host: checkpoints, caches, and result directories may be restored under the repository’s `outputs/` tree or supplied through environment overrides.

7 Evaluation

The evaluation should answer six questions tied directly to the paper’s main claim:

1. Does CS4M emit alerts at event arrival, before state update?
2. What event-level and node-level detection quality is observed on the current Linux, FreeBSD, Android, and Windows artifacts?
3. What RSS and throughput are observed during online scoring, and which source produced each measurement?
4. Does E4 dual target-case scoring expose calibrated, explainable alert evidence?
5. Do cross-OS semantic audits reveal portability strengths and limitations?
6. Can the evidence be reproduced without using test labels at runtime?

Algorithm 1: CS4M real-time event processing**Input:** event stream $\mathcal{E}$, embedding table, OFSM state table $\mathbf{S}_t, \pi_t$, calibrated thresholds, suppression cache**Output:** event alerts and node-level evidence

1. For each event $e_t = (s_t, d_t, a_t, \tau_t, c_t^s, c_t^d)$ in timestamp order:
2. Look up $\mathbf{e}_{s_t}$, $\mathbf{e}_{d_t}$, and $\mathbf{e}_{a_t}$.
3. Read pre-update logical states via π_t : $\mathbf{h}_{s_t}^{t-}$ and $\mathbf{h}_{d_t}^{t-}$.
4. Construct $\mathbf{x}_t$ using Eq. 38.
5. Select κ_t and $\mathbf{y}_t$ using Eqs. 39–41.
6. Predict $\hat{\mathbf{y}}_t$ with the target-case-specific E4 head in Eq. 43.
7. Compute ℓ_t, p_t , and σ_t .
8. Retrieve θ_{g_t} and apply pair-only suppression.
9. Emit event alert and node evidence.
10. For s_t and d_t , run OFSM copy-on-write if the mapped slot is shared.
11. Update the resulting physical slots with $\mathbf{z}_t$.
12. Periodically merge compatible physical slots satisfying Eq. 31.

Figure 1: CS4M’s online detection order. The model raises alerts before writing the current event into state memory. E4 dual scoring chooses a target-case specific low-rank head before thresholding, and OFSM controls memory growth.

This draft uses an evidence-lock workflow before writing final evaluation claims. The current locked tables live in ‘paper_rewriting_output/evidence_lock_topk_table.csv’, ‘paper_rewriting_output/evidence_lock_runtime_table.csv’, and ‘paper_rewriting_output/evidence_lock_optc051_summary.csv’. They are paper-facing inputs, not runtime policy files. Ground truth is used only after streaming inference has produced alert outputs.

OPTC_051 canonical-aware evidence. For the current Windows result, the locked post-stream audit reports 1366 total alerts with 208 TP and 1158 FP under canonical-aware event evaluation. At the entity level, it hits 66/114 original GT entities and 19/67 unique compact GT nodes. Coverage is strongest for netflow nodes, where the audit reports 49/49 original GT hits; process and file coverage are 15/59 and 2/6, respectively. These numbers are evaluation evidence only and must not be used to construct runtime thresholds or rescue rules.

8 Case Studies and Cross-OS Semantic Analysis

CS4M’s semantic layer is intentionally OS-aware. A provenance event has the same abstract form across datasets, but the runtime-visible fields that carry meaning differ substantially across FreeBSD, Linux, Android, and Windows. This section uses post-hoc semantic audits to explain portability and risk. These audits inspect existing node caches and metadata after experiments have run; they do not generate runtime rules, thresholds, or rescue policies.

FreeBSD and Linux. CADETS and THEIA are closest structurally: both expose POSIX-style process commands, paths, and network flows. Their global process generic ratios are

low in the audit, but their GT process generic ratios are higher. This does not mean the runtime detector used labels; it means the post-hoc audit found that some attack-relevant commands landed in fallback-like roles such as `process_other`. For netflows, both systems preserve exact endpoint detail for most nodes even when a broad IP-scope token is present. This supports CS4M’s cross-OS design choice: keep a shared event-arrival scoring model, but audit OS-specific tokenizer collapse separately.

Android. ClearScope differs from POSIX server datasets because Android processes are often package names or native system utilities, and files are dominated by app private storage, cache, `/proc`, `/dev`, and Android-specific state paths. The audit shows that ClearScope process nodes are not dominated by literal unknown tokens, but native process families and common system utilities can still create semantic collisions. ClearScope netflow is the most conservative case: runtime semantics fold netflows into a fixed token by default, unlike the THEIA, CADETS, and OpTC settings that retain exact or remote endpoint detail. This is a limitation of the current semantic extractor, not a threshold issue.

Windows. OpTC requires the largest semantic shift because Windows exposes process image names, command lines, registry paths, file families, extensions, and network service information. The Windows semantic audit motivates safe lexical command features: for example, command lines can expose encoded commands, URLs, UNC paths, scheduled-task queries, or LOLBin usage without storing full raw command strings. Windows netflow semantics also preserve remote endpoint and service detail. These design choices show why CS4M’s semantic layer cannot be a single hard-coded POSIX tokenizer; portability comes from applying the same online scoring contract to OS-specific runtime-visible semantics.

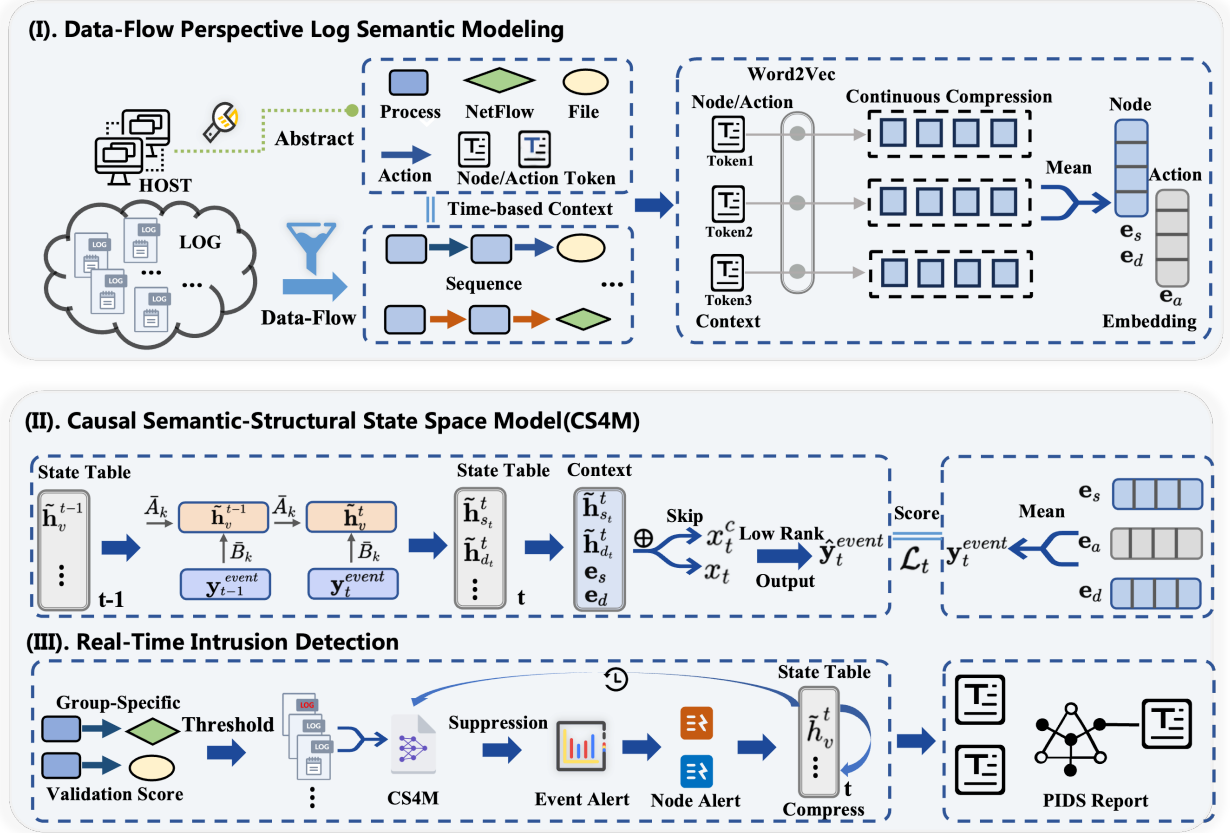


Figure 2: Overview of CS4M. The pipeline first converts host logs into data-flow provenance events and runtime-visible node/action semantics, then learns compact node and action embeddings. During online inference, CS4M reads pre-update source and destination states, constructs an E4 dual semantic prediction context, scores the residual before updating state, and emits event and node alert evidence. OFSM compresses active state storage after alerting; ground truth is used only later for evaluation.

9 Discussion and Limitations

CS4M is an alerting system, not a full attribution engine.

The paper’s primary contribution is event-arrival online alerting. CS4M emits event and node evidence immediately, but it does not attempt to reconstruct a complete attack graph or optimize attribution quality in the same sense as attribution-focused PIDSs. Graph reconstruction remains useful after an alert: an analyst can start from CS4M’s event/node evidence and then run graph expansion or forensic reconstruction to explain broader campaign context.

E4 dual is the paper method. The method section describes the E4 dual path: compact semantic state, target-case-specific low-rank residual scoring, score-before-update alerting, OFSM, validation-only calibration, and pair-only suppression. Other explored variants are not presented as paper contributions. This keeps the design claim testable and avoids mixing exploratory policies with the deployable path.

Semantic portability requires audit discipline. The same event-arrival online contract can be used across OSes, but tokenizers must respect each OS’s observable semantics. POSIX

paths, Android packages, Windows command lines, registry keys, and netflow endpoints carry different signals. The semantic audits are therefore part of the engineering discipline: they identify collapse and missing-detail risks after the fact. They must not be converted into label-derived runtime rules without retraining and validation.

Current evidence risks. Two evaluation risks remain visible in the draft. First, ClearScope E5 top- k must be recomputed under the 51-GT canonical-aware mapping before the final table can make a locked top- k claim. Second, RSS and throughput rows currently mix source types such as online-minimal profiling, deploy/test scoring, and log profiling. The table reports the source status explicitly, but final cross-dataset efficiency claims should wait until these measurement sources are normalized or clearly separated.

10 Related Work

Graph-first provenance IDS and investigation. Provenance graphs are the dominant abstraction for host-level APT detection because they preserve causal relations among processes, files, and network entities. UNICORN studies runtime

Dataset	top100	top200	top500	top1000	top3000	top5000	full pool
THEIA_E3	34/66	42/158	42/458	42/958	42/2958	42/4958	54/28692
CADETS_E3	32/68	32/168	34/466	34/966	43/2957	47/4953	53/6050
CLEARSCOPE_E3	1/99	1/199	3/497	13/987	40/2960	40/3297	40/3297
CLEARSCOPE_E5	Paper TODO: recompute top-k under 51-GT canonical-aware mapping						14/377
THEIA_E5	2/98	2/198	3/497	10/751	10/751	10/751	10/751
OPTC_051	3/97	3/197	17/377	17/377	17/377	17/377	17/377

Table 2: Locked node-pool top- k TP/FP evidence for the current paper draft. Rows marked with ‘Paper TODO’ are intentionally not final claims. ClearScope E5 currently has a source conflict: the dual-channel audit reports 14/377 full-pool TP/FP, while the default result-directory top- k file reports zero TP under its current mapping. The final paper must replace this placeholder with a 51-GT canonical-aware top- k recomputation.

Dataset	Selected source	Event TP/FP	Node TP/FP	RSS MB	events/s	Source status
THEIA_E3	E4 current best	152075/15311	54/28692	154.21	3450.39	deploy/test scoring
CADETS_E3	E4 best top- k	2397/8992	53/6050	89.92	3897.52	recorded deploy/test scoring
CLEARSCOPE_E3	V31 FP guard	413/9028	40/3297	73.94	3543.37	online minimal
CLEARSCOPE_E5	dual-channel audit	15/394	14/377	103.95	3363.85	Paper TODO: normalize source
THEIA_E5	v5c0 plus policy	11/1300	10/751	152.93	6291.01/6481.20	log/profiling
OPTC_051	Windows v1.3c full	208/1158	17/377	Paper TODO	Paper TODO	extract from metrics/profiling

Table 3: Current runtime and alert-quality evidence lock. These rows mix different speed/RSS sources and therefore should not be used for final cross-dataset efficiency comparison until the source column is reconciled. OPTC_051 already has canonical-aware event and GT-kind evaluation, but its runtime RSS and speed should be extracted from ‘metrics.json’, ‘rss_breakdown_online_event_core.json’, and ‘profiling.csv’ before final submission.

provenance-based APT detection under graph growth, while KAIROS and FLASH use whole-system provenance and graph representation learning for practical detection and investigation [1, 4, 6]. These systems are complementary to CS4M: they demonstrate the value of provenance context, but their computational object is still a graph or graph-derived representation. CS4M instead makes the arriving event the alerting unit.

Segmentation and window-based online PIDS. TAPAS is the closest related online framing because it explicitly targets efficient online APT detection. It reduces provenance graph cost through task-guided process provenance graph segmentation and low-dimensional process representations [5]. This is a strong deployment-oriented direction, but it still detects over active task subgraphs. CS4M makes a stricter design choice: the current event itself is scored before it updates detector state. The distinction is not that segmentation is wrong; it is that segmentation and event-arrival alerting answer different timing questions.

Attribution and triage systems. ORTHRUS targets attribution quality, and NoDoze targets alert triage and alert fatigue [7, 3]. These goals remain important after an alert is raised. CS4M’s node evidence is intended to make immediate alerts inspectable, but it is not a substitute for full attribution. A practical deployment could use CS4M to reduce alert latency and then use graph reconstruction or attribution systems to expand the investigation.

Runtime correlation and lightweight state models. HOLMES correlates suspicious information flows into higher-level attack evidence [2]. CS4M differs by avoiding rule-centered correlation as the primary inference object. It uses compact semantic state and low-rank residual scoring to make per-event inference feasible. The state-space notation draws on efficient sequence-modeling ideas such as S4/S4D [10, 11], but CS4M’s novelty is not a new generic state-space model. The contribution is the deployment contract: score each provenance event at arrival time, before state update, while preserving explainable evidence.

11 Conclusion

CS4M changes the online detection unit for provenance intrusion detection from a completed graph, window, segment, or periodic inference batch to the individual arriving event. It does so with compact semantic state, E4 dual residual scoring, score-before-update causality, OFSM-bounded active memory, and immediate event/node evidence. The current draft keeps unresolved evidence risks visible rather than converting them into unsupported claims. This is the core argument for deployable online PIDS: the log stream is not enough; the alert semantics must also be online.

A Theoretical Notes

This appendix records the mathematical properties used in the method section. These notes are intended to support the design

Dataset	Kind	Generic ratio	GT generic ratio	Exact-IP ratio	Main diagnostic signal
THEIA_E3	process	0.0006	0.2917	0.0000	Rare GT processes fall into fallback roles.
THEIA_E3	netflow	1.0000	1.0000	0.8154	Scope is coarse, but most endpoints retain exact IP.
CADETS_E3	process	0.0003	0.2500	0.0000	FreeBSD payload-like commands can collapse into process_other.
CADETS_E3	netflow	1.0000	1.0000	0.9983	Netflow is generic by scope but endpoint detail is preserved.
CLEARSCOPE_E3	file	0.0000	0.0000	0.0000	Android file semantics are detailed for E3.
CLEARSCOPE_E5	file	0.0148	0.0500	0.0000	Some Android file paths still collapse into generic buckets.
CLEARSCOPE_E5	netflow	1.0000	0.0000	0.0000	Runtime netflow semantics are fully folded by default.

Table 4: Representative post-hoc semantic audit signals. GT columns are diagnostic only and are not used by the runtime detector. The table highlights why cross-OS portability requires tokenizer audits even when the online scoring algorithm is unchanged.

claims; empirical effectiveness must be evaluated experimentally.

A.1 Zero-Order Hold Discretization

Proposition 1. For the continuous-time linear state-space model

$$\dot{\mathbf{h}}(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}\mathbf{u}(t), \quad (54)$$

with constant input $\mathbf{u}(t) = \mathbf{u}_k$ on $[t_k, t_{k+1})$, the exact discrete update is Eq. 11.

Proof. By variation of constants,

$$\mathbf{h}(t_{k+1}) = e^{\Delta_k \mathbf{A}} \mathbf{h}(t_k) + \int_0^{\Delta_k} e^{(\Delta_k - \tau) \mathbf{A}} \mathbf{B} \mathbf{u}_k d\tau. \quad (55)$$

Taking $\bar{\mathbf{A}}_k = e^{\Delta_k \mathbf{A}}$ and $\bar{\mathbf{B}}_k = \int_0^{\Delta_k} e^{(\Delta_k - \tau) \mathbf{A}} \mathbf{B} d\tau$ gives Eq. 11. If $\mathbf{A}$ is invertible, the integral equals $\mathbf{A}^{-1}(e^{\Delta_k \mathbf{A}} - \mathbf{I})\mathbf{B}$. This is the standard zero-order hold discretization used by linear systems and diagonal SSMs [10, 11].

A.2 OFSM Memory Bound

Proposition 2. OFSM state memory is bounded by the active-state capacity and physical slot count rather than the historical node count.

Proof. At time t , dense historical storage requires $|\mathcal{V}_{1:t}|$ state vectors and therefore $\mathcal{M}_{\text{dense}}(t) = |\mathcal{V}_{1:t}|db$. OFSM maintains states only for $\mathcal{A}_t$, where $|\mathcal{A}_t| \leq K_s$. It then maps logical active nodes to P_t physical slots, with $P_t \leq |\mathcal{A}_t|$. Thus the vector memory is $P_t db$, plus mapping overhead $\mathcal{M}_{\text{index}}$, yielding Eq. 24. Since $P_t \leq K_s$, the asymptotic state-vector memory is $O(K_s d)$ and does not grow linearly with $|\mathcal{V}_{1:t}|$. This does not mean memory is zero or independent of workload; it means the online state memory is governed by active capacity and physical slots, not by the total number of historical nodes observed in the log stream.

A.3 Copy-on-Write Isolation

Proposition 3. OFSM copy-on-write prevents a node-specific update from modifying the readable state of other logical nodes sharing the same physical slot.

Proof. Suppose nodes in $\mathcal{C}_p^t$ share slot p , and $v \in \mathcal{C}_p^t$ must be updated. If $|\mathcal{C}_p^t| > 1$, OFSM creates a new slot p' with $s_{p'}^t = s_p^t$, remaps $\pi_t(v) = p'$, and removes v from $\mathcal{C}_p^t$. The

update is applied only to $s_{p'}$. For any $u \in \mathcal{C}_p^t \setminus \{v\}$, $\pi_t(u) = p$ remains unchanged, so its readable state remains s_p^t . Therefore the update to v cannot alter the readable state of other nodes in the former cluster.

A.4 Score-Before-Update Causality

Proposition 4. CS4M’s score-before-update order prevents an event from being used to explain itself through node memory.

Proof. The context $\mathbf{x}_t$ is constructed from $\mathbf{h}_{s_t}^{t-}$ and $\mathbf{h}_{d_t}^{t-}$, which are read before $\mathbf{z}_t$ is written. The prediction $\hat{\mathbf{y}}_t$ and score σ_t are functions of $\mathbf{x}_t$, κ_t , and model parameters. Since the update to the physical state slots occurs only after scoring, $\mathbf{z}_t$ cannot influence $\mathbf{x}_t$ through the state table for the same event. Thus the event cannot reduce its own residual by first modifying memory.

References

- [1] Xueyuan Han, Thomas Pasquier, Adam Bates, James Mickens, and Margo Seltzer. UNICORN: Runtime Provenance-Based Detector for Advanced Persistent Threats. In *NDSS*, 2020.
- [2] Sadegh M. Milajerdi, Rigel Gjomemo, Birhanu Eshete, R. Sekar, and V. N. Venkatakrishnan. HOLMES: Real-time APT Detection through Correlation of Suspicious Information Flows. In *IEEE Symposium on Security and Privacy*, 2019.
- [3] Wajih Ul Hassan, Shengjian Guo, Ding Li, Zhengzhang Chen, Kangkook Jee, Zhichun Li, and Adam Bates. NoDoze: Combatting Threat Alert Fatigue with Automated Provenance Triage. In *NDSS*, 2019.
- [4] Zijun Cheng, Qiujian Lv, Jinyuan Liang, Yan Wang, Degang Sun, Thomas Pasquier, and Xueyuan Han. KAIROS: Practical Intrusion Detection and Investigation using Whole-system Provenance. In *IEEE Symposium on Security and Privacy*, 2024.
- [5] Bo Zhang, Yansong Gao, Changlong Yu, Boyu Kuang, Zhi Zhang, Hyoungshick Kim, and Anmin Fu. TAPAS: An Efficient Online APT Detection with Task-guided Process Provenance Graph Segmentation and Analysis. In *USENIX Security Symposium*, 2025.

- [6] Mati Ur Rehman, Hadi Ahmadi, and Wajih Ul Hassan. Flash: A Comprehensive Approach to Intrusion Detection via Provenance Graph Representation Learning. In *IEEE Symposium on Security and Privacy*, 2024.
- [7] Baoxiang Jiang, Tristan Bilot, Nour El Madhoun, Khaldoun Al Agha, Anis Zouaoui, Shahrear Iqbal, Xueyuan Han, and Thomas Pasquier. ORTHRUS: Achieving High Quality of Attribution in Provenance-based Intrusion Detection Systems. In *USENIX Security Symposium*, 2025.
- [8] Tristan Bilot, Baoxiang Jiang, Zefeng Li, Nour El Madhoun, Khaldoun Al Agha, Anis Zouaoui, and Thomas Pasquier. Sometimes Simpler is Better: A Comprehensive Analysis of State-of-the-Art Provenance-Based Intrusion Detection Systems. In *USENIX Security Symposium*, 2025.
- [9] Tristan Bilot and Thomas Pasquier. Toward Practical and Usable Provenance-based Intrusion Detection Systems. *USENIX ;login: Online*, 2025.
- [10] Albert Gu, Karan Goel, and Christopher Ré. Efficiently Modeling Long Sequences with Structured State Spaces. In *International Conference on Learning Representations*, 2022. <https://arxiv.org/abs/2111.00396>.
- [11] Albert Gu, Ankit Gupta, Karan Goel, and Christopher Ré. On the Parameterization and Initialization of Diagonal State Space Models. In *Advances in Neural Information Processing Systems*, 2022. <https://arxiv.org/abs/2206.11893>.